

SENTIMENT ANALYSIS USING NLP
PROJECT DOCUMENTATION
INTERSHIP TASK 02 – AI / ML

Name: Shazmeen

Program: AI / ML Internship

1. Introduction:

Sentiment analysis is an NLP technique used to find out whether a piece of text expresses a positive or negative opinion. For this task, I worked on movie reviews and built a simple sentiment classification model. The project starts with raw review text, cleans the text, converts it into numerical features and then uses a machine learning model to predict the sentiment. For the project, I used the IMDb movie review dataset and trained a Logistic Regression model.

2. Project Objective:

The main objective of this project was to build a basic NLP model that can classify movie reviews into **positive** and **negative** sentiments.

Through this project, I worked on the following steps:

- Loading and exploring the dataset
- Cleaning the review text
- Removing unnecessary words and characters
- Applying lemmatization
- Extracting features using TF-IDF
- Training a classification model
- Making predictions
- Evaluating the model using different metrics

The project also helped me understand how text data can be prepared and used in a machine learning classification problem.

3. Dataset:

For this project, I used the **IMDb movie review dataset**.

The dataset contains movie reviews along with their sentiment labels. The two labels used in the dataset are:

- **0 – Negative**
- **1 - Positive**

The dataset contains:

- **25,000 training reviews**
- **25,000 testing reviews**

I used the training data to train the model and the testing data to check how well the model performs on unseen reviews.

4. Dataset Exploration:

Before starting the preprocessing, I checked the dataset to understand its structure.

I checked:

- Number of training samples
- Number of testing samples
- Dataset columns
- Sentiment labels
- Example positive and negative reviews

The dataset contains a **Text** column for the review and a **Label** column for the sentiment.

The dataset is also balanced, with both positive and negative reviews available in the training and testing sets.

5. Project Workflow:

The complete project follows a simple NLP and machine learning workflow:

IMDb Dataset --> Data Exploration --> Text Preprocessing --> TF-IDF Feature Extraction -> Logistic Regression --> Sentiment Prediction --> Model Evaluation

First, the IMDb dataset was loaded and explored. After that, the raw reviews were cleaned using different text preprocessing techniques. The cleaned reviews were then converted into numerical features using TF-IDF. These features were used to train the Logistic Regression model. After training, the model was used to predict the sentiment of test reviews. Finally, the predictions were evaluated using different performance metrics.

6. Text Preprocessing:

Raw text usually contains unnecessary information that can affect the model. Therefore, I cleaned the reviews before using them for training.

The following preprocessing steps were used in the project.

1. Lowercasing:

All the review text was converted to lowercase.

For example:

This Movie Was Great

becomes:

This movie was great

This makes the text more consistent.

2. Removing HTML Tags:

Some IMDb reviews contain HTML tags. These tags are not useful for sentiment classification, so they were removed.

3. Removing Unnecessary Characters:

Unnecessary characters and numbers were removed from the reviews. This helped keep the text cleaner.

4. Stopword Removal:

Common English words were removed using NLTK's stopwords list. This helps reduce words that usually do not provide much information for the classification task.

5. Lemmatization:

I used WordNet Lemmatizer to convert words into their basic form. This helps group different forms of words together.

7. Feature Extraction Using TF-IDF:

After cleaning the reviews, I needed to convert the text into a format that the machine learning model could understand. For this purpose, I used **TF-IDF (Term Frequency-Inverse Document Frequency)**. TF-IDF gives numerical values to words based on how important they are in the reviews. Words that are common in a particular review but less common across the dataset can get higher importance. I used **TfidfVectorizer** with a maximum of **5,000 features**. The training

reviews were used to fit the vectorizer, and the same vectorizer was then used to transform the testing reviews.

8. Model Development:

For sentiment classification, I used **Logistic Regression**. Logistic Regression is a classification algorithm that can be used to predict categories. It works well with numerical features such as TF-IDF features, which is why it was selected for this project.

The model development process was:

1. Prepare the cleaned training reviews.
2. Convert the reviews into TF-IDF features.
3. Prepare the sentiment labels.
4. Train the Logistic Regression model.
5. Use the trained model to predict the sentiment of test reviews.

The model predicts whether a review belongs to the positive or negative sentiment class.

9. Model Evaluation:

To check the performance of the model, I used the four evaluation metrics required for the task:

- Accuracy
- Precision
- Recall
- F1-Score

The model produced the following results:

Accuracy: 87.92%

Precision: 87.73%

Recall: 88.17%

F1-Score: 87.95%

These results show that the model was able to classify most of the movie reviews correctly.

```
... Accuracy : 0.8792
    Precision: 0.8773
    Recall   : 0.8817
    F1-Score  : 0.8795
```

10. Classification Report:

The classification report showed almost the same performance for both positive and negative reviews. For the negative class, the precision, recall and F1-score were around **0.88**. For the positive class, these values were also around **0.88**. This shows that the model performed fairly similarly on both sentiment classes. The overall accuracy was **0.88**, which is the same as approximately **87.92%**.

...		precision	recall	f1-score	support
	Negative	0.88	0.88	0.88	12500
	Positive	0.88	0.88	0.88	12500
	accuracy			0.88	25000
	macro avg	0.88	0.88	0.88	25000
	weighted avg	0.88	0.88	0.88	25000

11. Confusion Matrix

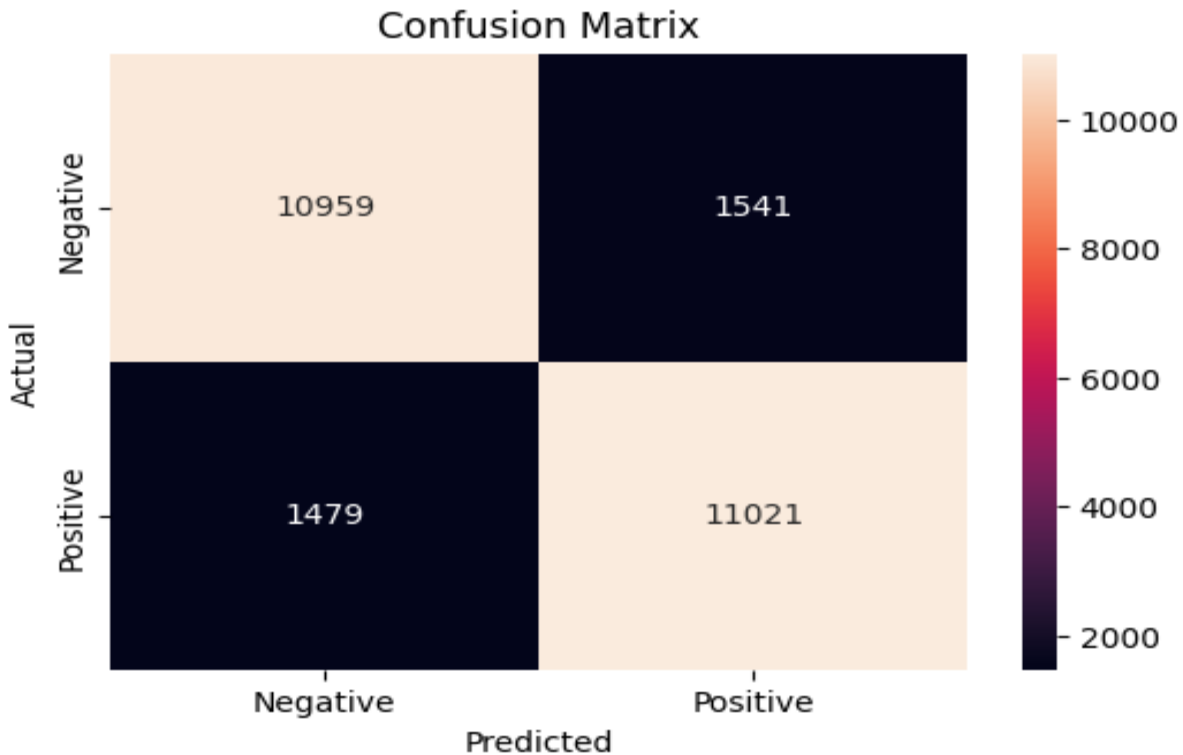
The confusion matrix obtained from the model was:

...	[[10959 1541]
	[1479 11021]]

The results can be understood as follows:

- **10,959** negative reviews were correctly predicted as negative.
- **1,541** negative reviews were predicted as positive.
- **1,479** positive reviews were predicted as negative.
- **11,021** positive reviews were correctly predicted as positive.

So, the model correctly classified **21,980 out of 25,000** test reviews.



12. Practical Applications:

Sentiment analysis can be used in many real-world situations.

- **Customer Feedback:**
Companies can analyze customer feedback to find out whether customers are satisfied or dissatisfied.
- **Product Reviews:**
Online stores can use sentiment analysis to analyze large numbers of product reviews and understand customer opinions.
- **Marketing:**
Marketing teams can analyze customer responses to understand how people react to products or campaigns.
- **Customer Support:**
Customer support teams can identify negative feedback and focus on complaints that need attention.
- **Social Media Feedback:**
Sentiment analysis can also be used to understand people's opinions and reactions on social media.

13. Limitations:

There are some limitations in the current project. The model uses TF-IDF, which mainly represents words based on their importance in the dataset. Because of this, it may not fully understand the context of a sentence. For example, sarcasm or a review containing both positive and negative opinions can be difficult to classify correctly. Another limitation is that the model was trained on movie reviews. Its performance may be different when used on other types of text, such as tweets, customer complaints or product reviews.

14. Possible Improvements:

There are several ways in which this project can be improved in the future.

- Try other machine learning algorithms such as Naive Bayes or Random Forest.
- Tune the Logistic Regression parameters.
- Experiment with different TF-IDF settings.
- Use word embeddings.
- Try transformer-based models such as BERT.
- Test the model on other types of datasets.
- Compare the performance of multiple models.

These improvements could help the model handle more complex text and improve its performance.

15. Tools and Libraries Used:

The following tools and libraries were used in this project:

- **Python:**
Used as the main programming language for implementing the project.
- **Pandas:**
Used for handling and exploring the dataset.
- **NumPy:**
Used for numerical operations.
- **NLTK:**
Used for text preprocessing, stopword removal and lemmatization.
- **Scikit-learn:**
Used for TF-IDF feature extraction, Logistic Regression and model evaluation.
- **Matplotlib:**
Used for creating visualizations.
- **Seaborn:**

Used for displaying the confusion matrix.

16. How to Run the Project:

To run the project, follow these basic steps:

1. Install the required Python libraries.
2. Open the **Sentiment_Analysis_NLP.ipynb** notebook.
3. Run the notebook cells in order.
4. The notebook will load the dataset, preprocess the reviews, train the model and generate predictions.

17. Conclusion:

In this project, I built a sentiment analysis model using NLP. I started with the IMDb movie review dataset and cleaned the text using different preprocessing techniques. After preprocessing, I converted the reviews into numerical features using TF-IDF. I then trained a Logistic Regression model and evaluated its performance using Accuracy, Precision, Recall and F1-Score. The model achieved **87.92% accuracy** on the test dataset. Overall, this project helped me understand the basic NLP workflow and how text can be processed and used for a machine learning classification problem.